

УЧЕБНИК: АРХИТЕКТУРА БАНКОВСКОГО ИИ И ПРОЕКТИРОВАНИЕ DATA LAKE

**Фундаментальное руководство для бизнес-аналитиков и архитекторов
данных**

АБСТРАКТ

Данный учебник представляет собой исчерпывающую методологию проектирования глобальной архитектуры банковского Искусственного Интеллекта. В основе лежит концепция непрерывного алгоритмического сопровождения жизненного цикла клиента. Главный инструмент проектирования: Синапс (динамическая алгоритмическая связь между микросистемами). Результат обучения: способность аналитика сформировать нормализованный сегмент Data Lake, готовый для интеграции в ML Core.

ПРЕДИСЛОВИЕ

Классический транзакционный банкинг исчерпал потенциал роста. Переход к концепции алгоритмического сопровождения требует переосмысления архитектуры данных. Банк функционирует как единый цифровой организм. Потребность клиента должна распознаваться алгоритмом до момента прямого обращения. Для реализации данной задачи требуется создание Data Lake нового типа. Разрозненные базы данных департаментов заменяются нейронной связностью. Аналитики проектируют маршруты передачи данных, учитывающие правовые ограничения, финансовые риски и психологический комфорт клиента. Настоящее руководство предоставляет стандартизированный инженерный алгоритм для выполнения этой работы.

ГЛОССАРИЙ И ТЕРМИНОЛОГИЯ

- **Синапс (Synapse):** Утвержденный канал передачи данных между микросистемами. Состоит из условия активации, отправителя, получателя, юридического обоснования и целевого действия.
- **Триггер (Trigger):** Событие, инициирующее Синапс. Внешний триггер: покупка, изменение геолокации. Внутренний триггер: достижение финансовой цели, снижение баланса.
- **Привычка (Habit):** Циклично повторяющийся поведенческий паттерн клиента, зафиксированный базой данных.
- **Системное эхо (System Echo):** Алгоритмический прогноз. Оценка влияния текущего финансового или жизненного решения клиента на его устойчивость в будущем.
- **Радиус ответственности (Radius of Impact):** Географический или социальный периметр, в котором ИИ осуществляет поиск ресурсов (товаров, услуг, вакансий) для клиента.
- **HitL (Human-in-the-Loop):** Протокол обязательного участия человека. Применяется при высоких рисках для ручной валидации решений алгоритма.

ЧАСТЬ I. АРХИТЕКТУРА И ЭКОСИСТЕМА

Глава 1. Справочник микросистем (Узлы Data Lake)

Архитектура банка разделена на 11 функциональных микросистем (Узлов). Каждая команда проектирует собственный слой данных. Для исключения дублирования в машинном обучении вводится строгий справочник идентификаторов (ID). Использование сторонних наименований запрещено.

Таблица 1.1. Базовый справочник Узлов и Артефактов

ID Узла	Наименование	Зона ответственности	Итоговый Артефакт для Data Lake
:---	:---	:---	:---
NODE-01	Архитектура	Каркас системы, протоколы API, маршрутизация.	Схема графовой базы данных, API-спецификации.
NODE-02	Аудирование	Юридический контроль, соответствие НПА.	Матрица легальности (Data Lineage) для каждого тега.
NODE-03	R&D	Анализ трендов, технологическая разведка.	Реестр перспективных триггеров и внешних API.

| **NODE-04** | Анкетирование | Интерфейс эмпатии, логическое дерево диалога. | Дерево принятия решений (Decision Tree), NLP-промпты. |

| **NODE-05** | Инновации | Визуализация, стримы, транзакции в один клик. | Модели UI/UX реакций на срабатывание Синапса. |

| **NODE-06** | OKR | Метрики успеха, декомпозиция жизненных целей. | Библиотека финансовых графов (Цель → Шаги → Сроки). |

| **NODE-07** | IDP | Обучение, трекинг профессиональных навыков. | База компетенций и образовательных треков. |

| **NODE-08** | Бизнес-рост | Поддержка предпринимательства, B2B-переходы. | Карта переходов статусов (Физлицо → ИП → ТОО). |

| **NODE-09** | Уиш лист | База материальных и нематериальных желаний. | Классификатор товаров/услуг с ценовыми диапазонами. |

| **NODE-10** | Платформа услуг | Экспресс-заработок, гиг-экономика, ресурсы. | Реестр верифицированных микрозадач и работодателей. |

| **NODE-11** | Маркетинг | Программа лояльности, мотивация, навигация. | Матрица стимулирующих коэффициентов и кэшбэков. |

Глава 2. Принципы безопасности (Security-by-Design)

Архитектура банковского ИИ строится на парадигме нулевого доверия (Zero Trust). Алгоритмы ML Core обучаются и работают исключительно с деперсонализированными массивами данных. Проектирование связей (Синапсов) обязано подчиняться трем базовым уровням защиты.

2.1. Строгая токенизация и хеширование

Прямые идентификаторы клиента (ФИО, ИИН, номера счетов, контактные данные) физически изолированы в защищенном контуре (Vault). В Data Lake эти данные заменяются криптографическим токеном (**Token_ID**). Процесс токенизации является односторонним для алгоритмов аналитики. Математическая модель, анализирующая риск дефолта или предлагающая экспресс-заработок, оперирует абстрактным **Token_ID: 9A4B-77X** . Дешифровка токена обратно в ФИО происходит только на финальном этапе формирования UI-интерфейса в приложении клиента, где доступ имеет исключительно сам пользователь.

2.2. Изолированное кросс-модульное взаимодействие

Функциональные узлы (11 команд) не имеют доступа к полным профилям клиентов. Обмен данными между модулями происходит атомарными пакетами (Payload), содержащими только статусы и математические значения.

Пример логики взаимодействия: Узел **NODE-09** (Уиш лист) фиксирует

потребность в средствах. Он формирует зашифрованный сигнал для узла **NODE-10** (Платформа услуг). Передаваемый пакет выглядит как `{"Token_ID": "9A4B-77X", "Trigger": "TRG-09-001", "Deficit_KZT": 50000}`. Модуль **NODE-10** обрабатывает эту потребность и возвращает ID подходящих микрозадач. Личность клиента, история его болезней или состав семьи остаются скрытыми от алгоритма подбора задач.

2.3. Контроль доступа на основе ролей (RBAC в Data Lake)

Бизнес-аналитики, работающие над разметкой данных, лишены физического доступа к продуктивным базам клиентов. Работа ведется на синтетических выборках (Mock Data) или на полностью агрегированных статистических наборах. Право на анализ связок токенов и реальных юридических документов закреплено исключительно за командой **NODE-02** (Аудирование) для контроля легитимности работы ИИ.

Глава 3. Многоуровневая архитектура Data Lake (Medallion Architecture)

Данные не загружаются в систему хаотично. Для обеспечения качества машинного обучения (предотвращения эффекта GIGO: Garbage In, Garbage Out) архитектура хранилища разделена на три логических слоя абстракции. По мере продвижения через слои, данные повышают свою ценность и структуру.

3.1. Bronze Layer (Сырые данные - Raw Layer)

- **Суть:** Неизменяемый архив (Append-only). Сюда попадают данные в исходном виде.
- **Контент:** Неструктурированные многостраничные тексты законов, постановления регуляторов, сырые API-ответы от внешних партнеров, нефильТРованные логи кликов пользователей в супераппе.
- **Доступ и функции:** Доступ открыт только Data-инженерам и команде аудита (**NODE-02**). Слой служит страховочным архивом на случай сбоя парсинга. Запрещено использовать Bronze Layer для обучения ML-моделей или построения дашбордов.

3.2. Silver Layer (Очищенные данные - Cleansed Layer)

- **Суть:** Пространство нормализации и стандартизации. Рабочая среда для 11 функциональных команд.

- **Контент:** Данные из Bronze Layer извлекаются с помощью ИИ (Gemini) и превращаются в атомарные характеристики. Бюрократический текст переводится в JSON-массивы, числовые лимиты, булевы значения (True/False) и строгие теги категорий.
- **Доступ и функции:** На этом уровне аналитики проводят очистку от дубликатов, приводят таблицы к стандартам нормализации (1NF-3NF) и решают логические коллизии между ведомствами. Данные Silver Layer используются для BI-аналитики и проверки гипотез.

3.3. Gold Layer (Агрегации и Синапсы - Curated Layer)

- **Суть:** Слой бизнес-решений и исполнения логики. Готовый продукт для внедрения в алгоритмы.
- **Контент:** Полностью выверенные и юридически обоснованные графы взаимосвязей. Здесь находятся рассчитанные индексы `Priority_Score` и `Homeostasis_Risk`. Разрозненные таблицы превращаются в готовые маршруты (Синапсы) с привязанными к ним эмпатичными интерфейсными сообщениями (NLP_Prompts).
- **Доступ и функции:** Прямой источник данных (Single Source of Truth) для ML Core. Алгоритмы машинного обучения забирают данные исключительно из Gold Layer для совершения предиктивной аналитики, генерации рекомендаций и ведения тактичного диалога с клиентом.

ЧАСТЬ II. ИНЖЕНЕРИЯ ДАННЫХ И ИНТЕГРАЦИЯ ИИ

Глава 4. Интеграция Gemini в рабочее пространство (Apps Script)

Ручной ввод текста из стандартов регулятора исключен. Аналитики используют генеративный ИИ для автоматического парсинга документов. В качестве среды прототипирования используется Google Sheets.

Технический регламент развертывания кода:

1. Создать новую книгу Google Sheets.
2. Перейти во вкладку "Расширения" (Extensions), выбрать "Apps Script".
3. Вставить следующий рабочий код интеграции:

```
// Базовый скрипт для обработки данных через API Gemini
const API_KEY = 'ВАШ_API_КЛЮЧ_ЗДЕСЬ';
const MODEL_NAME = 'gemini-2.5-flash';

function AI_CLASSIFY(prompt_text, source_text) {
  const url = `https://generativelanguage.googleapis.com/v1
beta/models/${MODEL_NAME}:generateContent?key=${API_KEY}`;

  const payload = {
    contents: [{ parts: [{ text: `${prompt_text}\n${source
_text}` }]}],
    generationConfig: { responseType: "application/jso
n" }
  };

  const options = {
    method: 'post',
    contentType: 'application/json',
    payload: JSON.stringify(payload)
  };

  try {
    const response = UrlFetchApp.fetch(url, options);
    const json = JSON.parse(response.getContentText());
    return json.candidates[0].content.parts[0].text;
  } catch (e) {
    return `Error: ${e.message}`;
  }
}
```

1. Сохранить проект. Вернуться в таблицу и использовать формулу `=AI_CLASSIFY($A$1; B2)` для массовой обработки.

Глава 5. Библиотека системных запросов (Prompt Engineering)

Машинное обучение невозможно построить на хаотичных текстах. Чтобы алгоритм (ML Core) мог сопоставлять данные от 11 разных команд, эти

данные должны иметь идентичную структуру. Бизнес-аналитик использует генеративный ИИ (Gemini) как парсер, который читает многостраничные юридические и продуктовые документы и извлекает из них строгие математические фильтры.

Для достижения 100% точности мы применяем метод Chain-of-Thought (Цепочка рассуждений). Аналитик не просто просит ИИ "сделать выжимку", он задает ИИ жесткий алгоритм действий шаг за шагом.

5.1. Шаблон экстракции технических параметров (Для команд NODE-06 - NODE-10)

Этот промпт используется, когда у вас есть текстовый регламент (например, условия предоставления микрозайма или правила регистрации ИП), и вам нужно перевести его в машиночитаемый формат JSON.

Текст Промпта:

Системная роль: Data Engineer банковского ИИ.
Задача: Изучи документ продукта: [ВСТАВИТЬ ТЕКСТ РЕГЛАМЕНТА].
Инструкция по выполнению (Chain-of-Thought):
Шаг 1. Определи базовую категорию продукта (например: кредитование, страхование, гиг-задача).
Шаг 2. Выдели все обязательные числовые параметры (сроки в днях, суммы в тенге, возрастные лимиты).
Шаг 3. Изолируй стоп-факторы (условия, при которых услуга строго запрещена).
Вывод: Сформируй строгий JSON-объект с ключами "category", "numerical_params", "stop_factors". Исключи любые текстовые рассуждения вне JSON-объекта.

Практический пример:

Вы подаете на вход ИИ текст: *"Кредит доверия выдается гражданам РК от 21 года на сумму до 500 000 тенге. Не допускается выдача лицам с действующей просрочкой более 30 дней".*

Результат, который ИИ поместит в ячейку вашей таблицы: `{"category": "microloan", "numerical_params": {"min_age": 21, "max_amount_kzt": 500000}, "stop_factors": ["overdue_days_gt_30"]}`. Этот JSON готов для алгоритма.

5.2. Шаблон генерации эмпатичного интерфейса (Для команды NODE-04 Анкетирование)

Команда Анкетирования выполняет функцию "переводчика" с машинного языка на человеческий. Если ИИ-агент просто выведет на экран клиента

надпись `stop_factors: overdue_days_gt_30`, это вызовет отторжение. Задача команды NODE-04 — сгенерировать NLP-промпты (Natural Language Processing), которые мягко соберут нужную информацию.

Текст Промпта:

Системная роль: UX/NLP Архитектор банковского приложения.
Задача: Трансформировать сухой банковский стоп-фактор [ВСТАВИТЬ JSON СТОП-ФАКТОРА ИЗ ШАГА 5.1] в тактичный интерфейсный вопрос.
Инструкция по выполнению:
Шаг 1. Проанализируй суть стоп-фактора.
Шаг 2. Сформулируй вопрос, который органично впишется в диалог о жизненных планах клиента. Клиент не должен чувствовать допрос или дискриминацию.
Шаг 3. Сформулируй 3 варианта ответа (позитивный, нейтральный, негативный).
Вывод: Предоставь варианты диалога от лица финансового ассистента.

Глава 6. Мировые стандарты обработки (ETL и Нормализация)

Данные, полученные от Gemini в Главе 5, еще не готовы к загрузке в ML Core. Они находятся в статусе Bronze Layer. Массив необходимо перевести в Silver Layer (Очищенные данные). Аналитик обязан провести массив через 4 инженерные стадии обработки. Пропуск хотя бы одной стадии ведет к деградации (поломке) нейросети.

6.1. Очистка (Cleaning)

Суть: Удаление технического мусора, который мешает математическим операциям.

- Практика: Аналитик применяет скрипты или формулы для удаления лишних пробелов, скрытых символов табуляции, унификации дат (все даты переводятся в стандарт YYYY-MM-DD) и приведения всех текстовых тегов к нижнему регистру.

6.2. Нормализация (1NF - 3NF)

Суть: Обеспечение атомарности баз данных. Система не может искать закономерности, если в одной ячейке хранится смешанная информация.

- Практика:

- Первое правило нормализации (1NF): Одна ячейка содержит строго одно значение. Если услуга требует паспорта и справки о доходах, вы не пишете в ячейку "паспорт, справка". Вы создаете массив JSON или две отдельные строки.
- Второе и Третье правила (2NF, 3NF): Все столбцы в вашей таблице должны зависеть исключительно от первичного ключа (Trigger_ID). Вы не дублируете названия ведомств текстом (например, "Министерство Юстиции"), вы используете строгий ID "ORG-007".

6.3. Data Lineage (Происхождение данных)

Суть: В банковском ИИ ни один алгоритмический вывод не может быть сделан "из воздуха". Аудитор должен иметь возможность проследить путь любого решения до конкретного пункта закона или внутреннего приказа банка.

- Практика: Каждая строка вашей таблицы обязана содержать ссылку на регламент. Если вы заложили фильтр "возраст от 18 лет", в столбце Data Lineage должен стоять номер приказа банка, утверждающий этот лимит. Без этого ИИ не пройдет юридический аудит.

6.4. Оздоровление (Healing) и поиск коллизий

Суть: Самый сложный этап, где пересекается работа 11 разных команд. Возникают коллизии (конфликты логики).

- Практика: Команда NODE-11 (Маркетинг) заложила в базу автоматическое предложение премиальной кредитной карты всем клиентам с оборотом выше 1 млн тенге. Но команда NODE-02 (Аудирование) зафиксировала в базе НПА запрет на выдачу кредитов лицам в стадии банкротства. Процесс "Оздоровления" — это написание перекрестных проверок (скриптов), которые ищут такие противоречия. Если скрипт находит конфликт, строка помечается статусом HitL (Human-in-the-Loop), и аналитик вручную прописывает правило приоритета (в данном случае запрет NODE-02 отменяет предложение NODE-11).

ЧАСТЬ III. ПРОЕКТИРОВАНИЕ СИНАПТИЧЕСКОЙ СЕТИ

Глава 7. Архитектура рабочих листов (Data Lake Schema)

Data Lake — это не просто набор разрозненных таблиц Excel. Это единая графовая структура. Чтобы алгоритмы интеграции могли бесшовно сшить (join) таблицы 1000 аналитиков из 11 разных команд, утвержден жесткий стандарт столбцов (Схема). Рабочий лист обязан содержать 7 параметров. Изменение названий столбцов, добавление или удаление параметров без согласования с NODE-01 запрещено.

Ниже приведен детальный разбор архитектуры листа на практическом примере. Представим, что команда NODE-10 (Платформа услуг) заводит в базу задачу "Курьер выходного дня" для экспресс-заработка клиента.

Столбец A. **Trigger_ID** (Первичный ключ)

- Описание: Уникальный буквенно-цифровой идентификатор жизненного события или потребности. Служит якорем для объединения баз.
- Формат заполнения: `[ID_Узла] - [Тип] - [Порядковый_номер]` .
- Пример: `TRG-10-A045` (Триггер узла 10, тип A, номер 045).

Столбец B. **Source_Document** (Data Lineage)

- Описание: Ссылка на внутренний регламент банка или нормативно-право акт РК, регулирующий данную услугу.
- Формат заполнения: Текст постановления или гиперссылка на внутренний портал банка.
- Пример: `Приказ ДК-2024/45 п.3.1 (Правила платформы микрозадач)` .

Столбец C. **Category_Tag** (Классификатор)

- Описание: Главный агрегирующий тег. Позволяет машинному обучению быстро фильтровать массивы данных, не углубляясь в сложные JSON-структуры.
- Формат заполнения: Одно слово на английском языке из утвержденного справочника тегов.
- Пример: `GIG_ECONOMY` .

Столбец D. **Technical_Attributes** (Параметры услуги)

- Описание: Технические и финансовые параметры, описывающие услугу. Генерируется ИИ (глава 5.1).
- Формат заполнения: Строгий валидный JSON.

- Пример: `{"income_kzt": 15000, "duration_hours": 4, "required_tools": ["smartphone"]}`. (Алгоритм видит, что задача принесет 15 тысяч, займет 4 часа и требует смартфона).

Столбец Е. `Ecological_Filter` (Стоп-факторы)

- Описание: Критически важный столбец. Здесь прописываются жесткие ограничения, при которых синапс (связь) должен быть заблокирован. Защищает клиента от неуместных предложений.
- Формат заполнения: Строгий валидный JSON.
- Пример: `{"min_age": 18, "physical_labor": true}`. (Алгоритм понимает, что нельзя предлагать эту задачу несовершеннолетним или клиентам с тегом инвалидности).

Столбец F. `NLP_Prompt` (Пользовательский интерфейс)

- Описание: Текст, который увидит клиент в приложении. Перевод технических атрибутов в живую речь. Зона ответственности команды NODE-04 (Анкетирование).
- Формат заполнения: Текстовая строка с переменными.
- Пример: `У вас есть 4 свободных часа в эти выходные? Есть задача рядом с вами, которая принесет 15 000 тенге прямо на карту. Потребуется только смартфон.`

Столбец G. `Priority_Score` (Скоринг)

- Описание: Математический приоритет действия. Если у клиента сработали одновременно триггер покупки и триггер заработка, алгоритм должен решить, какой NLP_Prompt показать первым.
- Формат заполнения: Числовое значение (Float), рассчитывается по формуле.
- Пример: `3.85`.

Итоговый вид строки в базе данных аналитика:

Таблица представляет собой плоский массив стандартизированных данных, готовый к выгрузке в базу данных (например, PostgreSQL или MongoDB) для дальнейшей работы ядра ML Core.

Глава 8. Алгоритм Скоринга и Математика Гомеостаза

ИИ-агент одновременно обрабатывает сотни триггеров. Для выбора оптимального действия применяются две взаимосвязанные формулы.

Модель 1: Priority Score (Индекс приоритета действия)

Определяет, какую "удочку" предложить клиенту первой.

$$P = \frac{W_t \times R_c}{E_f}$$

- W_t (**Вес триггера**): Значимость события по шкале от 0.1 до 1.0. Устанавливается аналитиком.
- R_c (**Ресурсная емкость**): Оценка наличия инструментов у клиента (навыки, свободное время). Шкала от 1.0 до 5.0.
- E_f (**Экологический фактор**): Коэффициент риска (от 1.0 до 5.0). Наличие медицинских или правовых барьеров повышает делитель, снижая приоритет неуместной услуги.

Модель 2: Homeostasis Risk (Индекс разрушения гомеостаза)

Определяет риск дефолта жизненного сценария. Если H_r превышает критическую отметку, ИИ блокирует все транзакционные предложения и переводит клиента в режим "Спасение".

$$H_r = (D_v \times C_v) + P_p$$

- D_v (**Дефицит**): Процент нехватки средств на базовые потребности.
- C_v (**Кредитная нагрузка**): Отношение обязательных платежей к доходам.
- P_p (**Штрафной паттерн**): Наличие деструктивных привычек (например, систематическое обнуление баланса до зарплаты).

ЧАСТЬ IV. ПРАКТИЧЕСКИЕ СЦЕНАРИИ (USE CASES)

Глава 9. Сквозные примеры алгоритмического сопровождения

Для лучшего понимания взаимодействия 11 микросистем рассмотрим практические сценарии работы готового Data Lake.

Сценарий 1: Профилактика долговой ямы (Студент 19 лет)

1. **Триггер**: Клиент ищет новый ноутбук в супераппе (**NODE-05**). Баланс карты близок к нулю.
2. **Распознавание**: **NODE-09** (Уиш лист) фиксирует желание. Алгоритм рассчитывает H_r (Риск гомеостаза). Риск высокий: покупка в кредит приведет к дефолту.

3. **Синапс-реакция:** Вместо предложения кредита активируется **NODE-10** (Платформа услуг).
4. **Эмпатия:** **NODE-04** (Анкетирование) формирует тактичный пуш: *"Вижу, вы присматриваете мощный ноутбук. Хотите взять 2-3 задачи по тестированию сайтов на выходных? Оплата покроет 30% стоимости аппарата."*
5. **Итог:** Потребность удовлетворена без закредитованности. Жизненный гомеостаз сохранен.

Сценарий 2: Масштабирование экспертности (Специалист 28 лет)

1. **Триггер:** Регулярные поступления от фриланс-бирж на карту физлица (**NODE-03** фиксирует поведенческий паттерн).
2. **Распознавание:** Анализ показывает стабильный рост. **NODE-08** (Бизнес-рост) подготавливает пакет для регистрации статуса Индивидуального Предпринимателя.
3. **Синапс-реакция:** **NODE-07** (IDP) подбирает бесплатный микрокурс по налогам. **NODE-11** (Маркетинг) готовит бонус в виде бесплатного обслуживания счета на год.
4. **Эмпатия:** Клиент получает сообщение: *"Ваши навыки востребованы. Переход в статус ИП займет 1 минуту и легализует ваши доходы. Мы подготовили для вас бесплатную бухгалтерию и бонусный курс."*
5. **Итог:** Банк бесшовно перевел клиента из B2C в B2B сегмент, выступив партнером по росту.

ЧАСТЬ V. ТЕСТИРОВАНИЕ И РЕЛИЗ

Глава 10. Трехэтапная валидация

Перед конвертацией заполненных таблиц в исполняемый программный код проводится финальный аудит качества. База данных Silver Layer подвергается жесткому стресс-тестированию по трем векторам. Пропуск ошибки на этом этапе приведет к системным сбоям в ядре алгоритма (ML Core).

10.1. Функциональный аудит (Проверка алгоритмической целостности)

Цель данного этапа — исключить сбои маршрутизации данных между 11 микросистемами. Аналитики проверяют связность графа:

- **Поиск «Мертвых концов» (Dead Ends):** Каждое событие (Триггер) обязано иметь спроектированный Синапс-ответ. Если клиент достиг финансовой цели (сработал триггер `NODE-06`), но система не сформировала действие поддержки (например, предложение инвестиционного продукта от `NODE-08`), алгоритм зависнет. Тупиковые ветви недопустимы и бракуются автоматическим скриптом.
- **Поиск «Логических петель» (Infinite Loops):** Критическая ошибка проектирования. Возникает, когда Синапс узла А вызывает реакцию узла Б, которая, в свою очередь, снова активирует узел А. Архитектура должна быть однонаправленной (Directed Acyclic Graph).
- **Инструментарий:** Для тестирования аналитики запускают встроенные макросы Apps Script, которые симулируют прохождение данных по всем веткам дерева решений.

10.2. Юридический аудит (Data Lineage Verification)

Искусственный интеллект в банковском секторе не имеет права совершать предложения без юридического основания. Аудит легитимности — зона ответственности узла `NODE-02`.

- **Механика проверки:** Запускается парсер, верифицирующий заполненность Столбца В (`Source_Document`) для каждой транзакции.
- **Требования к валидации:** Ссылка обязана вести на действующий нормативно-правовой акт (НПА) или актуальный внутренний регламент банка. Если скрипт обнаруживает, что документ имеет статус *Deprecated* (отменен или устарел), весь ассоциированный Синапс блокируется до актуализации правовой базы. Данный аудит исключает риск штрафов со стороны государственных регуляторов.

10.3. Экологический аудит (Human-in-the-Loop Protocol)

Машинная логика слепа к человеческим эмоциям. Тексты, сгенерированные в столбце `NLP_Prompt`, проходят обязательную выборочную проверку (HitL) для обеспечения психологического комфорта клиента.

- **Механика HitL:** Случайная выборка (5-10% от общего объема) сгенерированных ИИ текстов передается независимой комиссии (Compliance & UX-редакторы).
- **Критерии оценки:** Эксперты проверяют отсутствие «триггеров тревожности» (Anxiety triggers), исключение дискриминационных

формулировок и соблюдение корпоративного Tone of Voice.

- **Результат аудита:** Если формулировка признается агрессивной или навязчивой (нарушает профессиональную этику), весь пул промптов данного узла возвращается на переработку и повторную генерацию.

Глава 11. Перевод в код (Logic to ML Core)

После успешного прохождения трехэтапной валидации данные переходят в статус Gold Layer. Плоские нормализованные таблицы экспортируются из Google Sheets и трансформируются в исполняемые структуры для ядра машинного обучения.

11.1. Трансформация плоских данных в многомерные массивы

Табличный формат удобен для работы аналитика, но непригоден для ML-моделей. Используется автоматизированный алгоритм конвертации табличной логики в многомерные массивы формата JSON.

Каждая строка таблицы преобразуется во вложенную иерархическую структуру, где триггер является корневым узлом, а экологические фильтры и NLP-сообщения — его ветвями.

Пример экспортируемого объекта:

```
{
  "trigger_id": "TRG-10-A045",
  "category": "GIG_ECONOMY",
  "execution_rules": {
    "priority_score": 3.85,
    "ecological_constraints": {"min_age": 18}
  },
  "action": {
    "target_node": "NODE-04",
    "ui_payload": "У вас есть 4 свободных часа..."
  }
}
```

11.2. Конвертация табличной логики в исполняемые правила

Структурированные JSON-файлы не просто хранятся, они преобразуются в логические операторы. Скрипты-конвертеры переводят условия из столбцов D (Технические атрибуты) и E (Экологические фильтры) в деревья принятия решений (Decision Trees) или Python-функции с логикой

IF/THEN/ELSE . Формулы приоритета (\$P\$) и риска гомеостаза (\$H_r\$) встраиваются непосредственно в математические пайплайны алгоритма.

11.3. Интеграция в ML Core и Цикл обратной связи

Итоговые JSON-файлы и сгенерированные скрипты передаются в IT-департамент через защищенные пайплайны CI/CD. Они загружаются в обучающую выборку ML Core.

С этого момента разрозненные таблицы превращаются в функционирующую архитектуру принятия решений. В реальном времени ML Core начинает сопоставлять поступающие данные профилей клиентов с заложенными правилами (Синапсами).

Цикл замыкается: аналитики получают доступ к аналитическим дашбордам, где отслеживают метрики конверсии — как часто срабатывает спроектированный ими Синапс и насколько эффективна предложенная ИИ «удочка». На основе этих данных аналитики возвращаются к Главе 4 для оптимизации параметров системы.

ЗАКЛЮЧЕНИЕ И ЧЕК-ЛИСТ АРХИТЕКТОРА

В результате прохождения данной методологии вы спроектировали и собрали центральное ядро банковского ИИ-агента. Этот алгоритм обучен:

1. Распознавать скрытые жизненные триггеры и поведенческие привычки клиента до наступления финансовых или социальных кризисов.
2. Выполнять строгий математический расчет приоритета действий (\$P\$) и риска разрушения жизненного гомеостаза (\$H_r\$).
3. Автоматически соблюдать нормативно-правовые акты (Data Lineage) и этические границы (Экологические фильтры), исключая выдачу токсичных или незаконных предложений.
4. Трансформировать многомерные технические параметры (Silver Layer) в эмпатичный и безопасный диалог с пользователем.

Функционал и экосистемная связность (Симбиоз проектов):

Созданный ИИ-агент работает не как набор изолированных баз данных, а как единая нейронная сеть, где 11 проектов функционируют в алгоритмическом симбиозе.

Функционал системы раскрывается через перекрестную маршрутизацию. Например, когда микросистема Уиш-лист (**NODE-09**) фиксирует новую материальную потребность, этот сигнал не остается внутри одного

модуля. Ядро передает импульс в модуль OKR (**NODE-06**) для декомпозиции цели на сроки и суммы. При обнаружении дефицита бюджета система блокирует кредитные предложения (снижая риск \$H_r\$) и активирует Платформу услуг (**NODE-10**) для подбора задач экспресс-заработка, параллельно подключая модуль IDP (**NODE-07**) для быстрой прокачки требуемых навыков.

Весь этот сложный бэкэнд-процесс обрабатывается интегратором Анкетирования (**NODE-04**), который переводит агрегированные данные 10 команд в один своевременный, тактичный пуш в интерфейсе супераппа (**NODE-05**). Инновации, маркетинг и поддержка бизнеса выступают единым защитным коконом, активируясь только тогда, когда это алгоритмически безопасно для клиента.

Перед выгрузкой вашего сегмента Data Lake в продуктивную среду ML Core проведите финальную самопроверку по чек-листу:

- ☐ Скрипт Apps Script интегрирован, парсинг данных через API Gemini стабилен.
- ☐ Характеристики извлечены из НПА без потери юридического смысла.
- ☐ Требования нормализации (атомарность ячеек 1NF-3NF) строго соблюдены.
- ☐ Источники происхождения (Data Lineage) указаны для 100% строк.
- ☐ Экологические фильтры прописаны, коллизии с предложениями устранены на этапе оздоровления.
- ☐ Формулы \$P\$ (Priority Score) и \$H_r\$ (Homeostasis Risk) настроены.
- ☐ Проектирование сквозных Синапсов завершено: каждый триггер имеет логичный маршрут решения.

Ваша работа формирует фундамент интеллекта системы. От качества созданных вами датасетов зависит уровень финансовой защиты и жизненного комфорта миллионов клиентов.

ПРИЛОЖЕНИЕ. РЕКОМЕНДУЕМАЯ ЛИТЕРАТУРА И АКАДЕМИЧЕСКИЕ МАТЕРИАЛЫ

Для углубленного изучения архитектурных принципов, заложенных в данной методологии, бизнес-аналитикам и архитекторам данных

рекомендуется ознакомиться со следующими фундаментальными трудами:

1. Инженерия данных и Архитектура баз данных (Data Engineering & Architecture)

- **Мартин Клеппман (Martin Kleppmann).** *«Высоконагруженные приложения. Программирование, масштабирование, поддержка» (Designing Data-Intensive Applications)*. Базовый труд для понимания проектирования надежных, масштабируемых и поддерживаемых распределенных систем хранения данных. Раскрывает логику построения графов и работу с коллизиями.
- **Билл Инмон (Bill Inmon), Мэри Лэшли.** *«Архитектура Data Lakehouse» (Building the Data Lakehouse)*. Фундаментальное объяснение концепции хранилищ нового поколения и многоуровневой архитектуры данных (Bronze, Silver, Gold Layers).
- **Авраам Сильбершатц (Abraham Silberschatz).** *«Основы баз данных» (Database System Concepts)*. Мировой академический стандарт изучения нормализации баз данных (1NF, 2NF, 3NF), обеспечения атомарности и целостности транзакций.

2. Системное мышление и проектирование процессов (Systems Thinking)

- **Донелла Медоуз (Donella Meadows).** *«Азбука системного мышления» (Thinking in Systems: A Primer)*. Обязательная литература для понимания концепции «гомеостаза», обратных связей (feedback loops), триггеров и системных задержек, применяемых при расчете риска (\$H_r\$).
- **Джин Росс (Jeanne W. Ross), Питер Вейл.** *«Архитектура предприятия как стратегия» (Enterprise Architecture as Strategy)*. Руководство по интеграции изолированных IT-модулей (у нас это 11 функциональных узлов) в единую экосистему на уровне бизнес-логики.

3. Искусственный Интеллект и Prompt Engineering

- **Официальная документация Google Gemini API.** Технические инструкции по интеграции генеративного ИИ посредством REST API, синтаксис настройки параметров генерации (temperature, response schema) и работа с форматом JSON.

- **Иэн Гудфеллоу (Ian Goodfellow), Йошуа Бенджио.** *«Глубокое обучение» (Deep Learning)*. Математическая база для понимания работы скрытых слоев нейросетей, расчета алгоритмических весов (Weights) и принципов машинного обучения.
- **Учебные материалы по NLP (Natural Language Processing).** Для узла **NODE-04**. Практики создания эмпатичных интерфейсов, извлечения тональности и построения логических деревьев диалога с клиентом.

4. Безопасность и Этика (Security & Ethics)

- **Росс Андерсон (Ross Anderson).** *«Инженерия безопасности» (Security Engineering)*. Настольная книга по проектированию устойчивых систем (Security-by-Design), защите от внутренних и внешних угроз, применению протоколов нулевого доверия (Zero Trust) и шифрования.
- **Материалы IEEE по этике автономных и интеллектуальных систем (Ethically Aligned Design).** Академические стандарты внедрения механизмов ручного контроля (Human-in-the-Loop), обеспечивающих защиту прав и психоэмоционального благополучия пользователя.